

OSSP-PRACTICAL-WEEK 1 Task 1 :

Develop a C program that demonstrates how a Linux operating system executes a command entered by a user by accepting a Linux command as input, creating a child process using the fork() system call, executing the command in the child process using an appropriate exec() system call, allowing the parent process to wait for the child using wait(), and displaying the Process ID (PID) of both the parent and child processes.

Source:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    char command[100];
    char *args[20];
    int i = 0;
    pid_t pid;

    printf("Enter Linux command: ");
    fgets(command, sizeof(command), stdin);

    char *token = strtok(command, " \n");

    while (token != NULL)
    {
        args[i++] = token;
        token = strtok(NULL, " \n");
    }

    args[i] = NULL;

    pid = fork();

    if (pid < 0)
    {
        perror("Fork failed");
        return 1;
    }

    else if (pid == 0)
    {
        printf("\nChild Process\n");

        if (pid < 0)
        {
            perror("Fork failed");
            return 1;
        }

        else if (pid == 0)
        {
            printf("\nChild Process\n");
            printf("Child PID: %d\n", getpid());

            execvp(args[0], args);

            perror("Command execution failed");
            exit(1);
        }

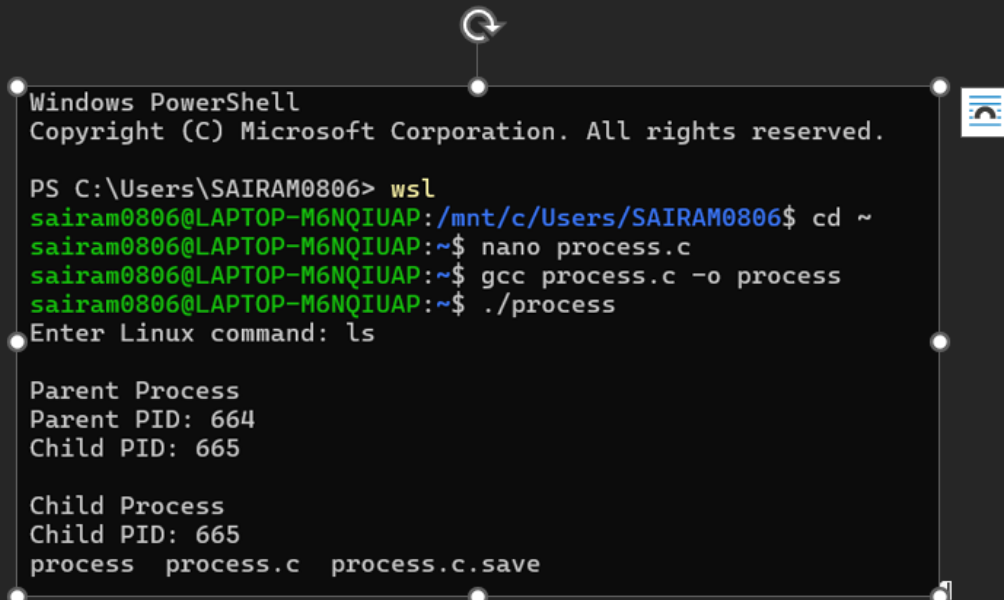
        else
        {
            printf("\nParent Process\n");
            printf("Parent PID: %d\n", getpid());
            printf("Child PID: %d\n", pid);

            wait(NULL);

            printf("\nChild process completed.\n");
        }

        return 0;
    }
}
```

Output:

A screenshot of a Windows PowerShell terminal window. The window title is "Windows PowerShell" and the copyright notice "Copyright (C) Microsoft Corporation. All rights reserved." is visible. The user is in the directory "C:\Users\SAIRAM0806" and has run the "wsl" command. The terminal shows the WSL prompt "sairam0806@LAPTOP-M6NQIUAP:/mnt/c/Users/SAIRAM0806\$". The user has run "cd ~", "nano process.c", "gcc process.c -o process", and "./process". The prompt "Enter Linux command: ls" is shown. The output of the "ls" command is displayed, showing "Parent Process", "Parent PID: 664", "Child PID: 665", "Child Process", "Child PID: 665", and "process process.c process.c.save".

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\SAIRAM0806> wsl
sairam0806@LAPTOP-M6NQIUAP:/mnt/c/Users/SAIRAM0806$ cd ~
sairam0806@LAPTOP-M6NQIUAP:~$ nano process.c
sairam0806@LAPTOP-M6NQIUAP:~$ gcc process.c -o process
sairam0806@LAPTOP-M6NQIUAP:~$ ./process
Enter Linux command: ls

Parent Process
Parent PID: 664
Child PID: 665

Child Process
Child PID: 665
process process.c process.c.save
```

Observation:

I learned how the Linux operating system creates a child process using `fork()` and executes a Linux command using the `execvp()` system call. I also understood how the parent process waits for the child process using `wait()` and how PID and PPID help identify the relationship between parent and child processes.

Task2: Using Linux terminal commands such as `uname`, `lscpu`, `lsblk`, `ps`, and `top`, investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the operating system abstracts and manages CPU, memory, storage, and I/O devices.

Output: Using command `uname -a`:

```
PS C:\Users\SAIRAM0806> wsl
sairam0806@LAPTOP-M6NQIUAP:/mnt/c/Users/SAIRAM0806$ uname -a
Linux LAPTOP-M6NQIUAP 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC
Thu Jun 18 21:54:43 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
```

Using command lscpu:

```
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ lscpu
Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
Address sizes:               48 bits physical, 48 bits virtual
Byte Order:                  Little Endian
CPU(s):                      12
On-line CPU(s) list:        0-11
Vendor ID:                   AuthenticAMD
Model name:                  AMD Ryzen 5 5600H with Radeon Graphics
CPU family:                  25
Model:                       80
Thread(s) per core:         2
Core(s) per socket:         6
Socket(s):                   1
Stepping:                    0
BogoMIPS:                    6587.45
Flags:                       fpu vme de pse tsc msr pae mce cx8 apic sep mtr
                             r pge mca cmov pat pse36 clflush mmx fxsr sse s
                             se2 ht syscall nx mmxext fxsr_opt pdpe1gb rdtsc
                             p lm constant_tsc rep_good nopl xtopology tsc_r
                             eliable nonstop_tsc cpuid extd_apicid tsc_known
                             _freq pni pclmulqdq ssse3 fma cx16 sse4_1 sse4_
                             2 movbe popcnt aes xsave avx f16c rdrand hyperv
                             isor lahf_lm cmp_legacy svm cr8_legacy abm sse4
                             a misalignsse 3dnowprefetch osvw topoext perfct
                             r_core ssbd ibrs ibpb stibp vmmcall fsgsbase bm
                             il avx2 smep bmi2 erms invpcid rdseed adx smap
                             clflushopt clwb sha_ni xsaveopt xsavec xgetbv1
                             xsaves clzero xsaveerptr arat npt nrrip_save tsc
                             _scale vmcb_clean flushbyasid decodeassists pau
                             sefilter pfthreshold v_vmsave_vmload umip vaes
                             vpcmlulqdq rdpid fsrm

Virtualization features:
Virtualization:              AMD-V
Hypervisor vendor:          Microsoft
Virtualization type:         full
Caches (sum of all):
L1d:                         192 KiB (6 instances)
L1i:                         192 KiB (6 instances)
L2:                           3 MiB (6 instances)
L3:                          16 MiB (1 instance)
NUMA:
NUMA node(s):                1
NUMA node0 CPU(s):          0-11
Vulnerabilities:
Gather data sampling:        Not affected
Ghostwrite:                  Not affected
Indirect target selection:    Not affected
Itlb multihit:               Not affected
L1tf:                        Not affected
Mds:                          Not affected
Meltdown:                    Not affected
Mmio stale data:              Not affected
Old microcode:                Not affected
Reg file data sampling:       Not affected
Retbleed:                     Not affected
Spec rstack overflow:         Vulnerable: Safe RET, no microcode
Spec store bypass:            Mitigation; Speculative Store Bypass disabled v
                               ia prctl
Spectre v1:                   Mitigation; usercopy/swaps barriers and __user
                               pointer sanitization
Spectre v2:                   Mitigation; Retpolines; IBPB conditional; IBRS_
                               FW; STIBP always-on; RSB filling; PBRSE-eIBRS N
                               ot affected; BHI Not affected
Srbds:                        Not affected
Tsa:                          Vulnerable: No microcode
Tsx async abort:              Not affected
Vmscape:                      Not affected
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ |
```

Using command lsblk:

```
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ lsblk
NAME
    MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda     8:0    0 356.9M  1 disk
sdb     8:16   0 159.4M  1 disk
sdc     8:32   0    1G    0 disk [SWAP]
sdd     8:48   0    1T    0 disk /mnt/wslg/distro
        /
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ |
```

Using command ps:

```
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ ps
  PID TTY          TIME CMD
  1015 pts/2    00:00:00 bash
  1163 pts/2    00:00:00 ps
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ |
```

Using command top:

```
top - 16:29:43 up 2:22, 1 user, load average: 0.00, 0.00, 0.00
Tasks: 24 total, 1 running, 23 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni, 100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 3580.6 total, 2843.9 free, 483.2 used, 328.5 buff/cache
MiB Swap: 1024.0 total, 1024.0 free, 0.0 used, 3097.4 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	21656	13176	9792	S	0.0	0.4	0:02.43	systemd
2	root	20	0	3180	2212	2072	S	0.0	0.1	0:00.08	init-systemd(Ub
7	root	20	0	3180	2036	1940	S	0.0	0.1	0:00.00	init
46	root	19	-1	50460	16224	15136	S	0.0	0.4	0:00.96	systemd-journal
95	root	20	0	25288	6580	5128	S	0.0	0.2	0:02.78	systemd-udev
116	systemd+	20	0	21352	13256	11128	S	0.0	0.4	0:00.39	systemd-resolve
122	systemd+	20	0	91044	7960	6992	S	0.0	0.2	0:00.50	systemd-timesyn
162	root	20	0	4248	2680	2428	S	0.0	0.1	0:00.11	cron
163	message+	20	0	9656	5368	4668	S	0.0	0.1	0:00.15	dbus-daemon
184	root	20	0	17980	8872	7844	S	0.0	0.2	0:00.33	systemd-logind
213	syslog	20	0	222524	5996	4604	S	0.0	0.2	0:00.24	rsyslogd
234	root	20	0	3128	1964	1832	S	0.0	0.1	0:00.02	agetty
261	root	20	0	107052	23176	13724	S	0.0	0.6	0:00.42	unattended-upgr
396	root	20	0	6708	4628	3840	S	0.0	0.1	0:00.02	login
442	sairam0+	20	0	20336	11580	9452	S	0.0	0.3	0:00.26	systemd
445	sairam0+	20	0	21184	3596	1868	S	0.0	0.1	0:00.00	(sd-pam)
471	sairam0+	20	0	6084	5360	3692	S	0.0	0.1	0:00.04	bash
623	root	20	0	3184	1124	980	S	0.0	0.0	0:00.00	SessionLeader
625	root	20	0	3200	1260	1108	S	0.0	0.0	0:00.05	Relay(631)
631	sairam0+	20	0	6084	5416	3696	S	0.0	0.1	0:00.28	bash
1009	root	20	0	3184	1124	980	S	0.0	0.0	0:00.00	SessionLeader
1010	root	20	0	3200	1260	1108	S	0.0	0.0	0:00.12	Relay(1015)
1015	sairam0+	20	0	6216	5456	3676	S	0.0	0.1	0:00.36	bash
1172	sairam0+	20	0	9308	5692	3508	R	0.0	0.2	0:00.43	top

Observation:

I learned how to use Linux commands to examine system hardware and running processes. I understood how the operating system manages CPU, storage, memory, and processes and provides these resources to applications through system services